Lie Groups and Lie Algebras for Reinforcement Learning-Based Robotics

Contents

1 Introduction

Robotic systems contain physical quantities whose configuration spaces are not ordinary Euclidean vector spaces.

Examples include

$$R \in \mathrm{SO}(3),$$

for three-dimensional orientation, and

$$T \in \mathrm{SE}(3),$$

for a rigid-body pose.

The objective of this document is to develop the mathematical machinery required to represent, manipulate, and learn such quantities in robotics and reinforcement learning.

The central objects are

$$\mathrm{SO}(3), \qquad \mathfrak{so}(3), \qquad \mathrm{SE}(3), \qquad \mathfrak{se}(3).$$

The principal maps are

$$\exp : \mathfrak{g} \to G$$

and

$$\log : G \to \mathfrak{g}.$$

These maps allow nonlinear robot configurations to be related to vector spaces in which numerical optimization and machine learning can be performed.

For reinforcement learning, the resulting structure is

robot state $\longrightarrow$ Lie-group representation $\longrightarrow$ Lie-algebra coordinates $\longrightarrow$ neural policy.

2 Mathematical Foundations

2.1 Vector Spaces

A vector space over $\mathbb{R}$ is a set V equipped with vector addition and scalar multiplication satisfying the usual vector-space axioms.

The Euclidean space

$$\mathbb{R}^n$$

is the primary example.

Vectors can be added:

$$x + y,$$

and multiplied by scalars:

$$\alpha x.$$

A key property of vector spaces is that global linear coordinates are available.

Many robotic quantities, however, do not possess this structure globally.

2.2 Groups

A group is a set G with a binary operation

$$\circ : G \times G \to G$$

satisfying:

1. Closure:
$$a \circ b \in G.$$

2. Associativity:
$$(a \circ b) \circ c = a \circ (b \circ c).$$

3. Identity:
$$e \circ a = a \circ e = a.$$

4. Inverse:
$$a \circ a^{-1} = a^{-1} \circ a = e.$$

Matrix multiplication provides an important example of a group operation.

2.3 Manifolds

A d-dimensional manifold is a space that locally resembles

$$\mathbb{R}^d.$$

A point on a manifold can therefore be described locally by d coordinates.
For example, the unit circle

$$S^1 = \left\{(x, y) \in \mathbb{R}^2 : x^2 + y^2 = 1\right\}$$

is a one-dimensional manifold.
Although it is embedded in $\mathbb{R}^2$, it has only one intrinsic degree of freedom.
Similarly,

$$\mathrm{SO}(3)$$

is a three-dimensional manifold.

2.4 Tangent Spaces

Let $\mathcal{M}$ be a manifold and let $x \in \mathcal{M}$.
The tangent space at x is denoted

$$T_x\mathcal{M}.$$

It contains all possible infinitesimal directions of motion through x.
For a Lie group G, the tangent space at the identity element e is particularly important:

$$\boxed{\mathfrak{g} = T_eG.}$$

The vector space $\mathfrak{g}$ is the Lie algebra associated with G.

3 Lie Groups

A Lie group is a group that is also a smooth manifold, with smooth group multiplication and inversion.

Let

$$G$$

be a Lie group.

The group operation is

$$(g_1, g_2) \mapsto g_1 g_2$$

and the inverse operation is

$$g \mapsto g^{-1}.$$

Important Lie groups in robotics include

$$\mathrm{SO}(2), \qquad \mathrm{SO}(3), \qquad \mathrm{SE}(2), \qquad \mathrm{SE}(3).$$

4 The Rotation Group $SO(3)$

The three-dimensional special orthogonal group is

$$\boxed{\mathrm{SO}(3) = \left\{ R \in \mathbb{R}^{3\times 3} : R^T R = I,\ \det R = 1 \right\}.}$$

Every element

$$R \in \mathrm{SO}(3)$$

represents a rotation in three-dimensional space.

4.1 Properties of Rotation Matrices

For

$$R \in \mathrm{SO}(3),$$

we have

$$R^T R = I.$$

Therefore

$$R^{-1} = R^T.$$

Also,

$$\det R = 1.$$

The identity is

$$I = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

The product of two rotations is another rotation:

$$R_1 R_2 \in \mathrm{SO}(3).$$

4.2 Degrees of Freedom

A general 3×3 matrix contains nine parameters.

The constraint

$$R^T R = I$$

imposes six independent constraints.

The determinant condition selects the proper rotations.

Consequently,

$$\boxed{\dim \mathrm{SO}(3) = 3.}$$

A three-dimensional orientation therefore has three degrees of freedom.

4.3 Action on Vectors

A vector

$$p \in \mathbb{R}^3$$

is rotated according to

$$p' = Rp.$$

The Euclidean norm is preserved:

$$\|Rp\|_{2\,2} = \|p\|_{2\,2}.$$

Indeed,

$$\|Rp\|_2{}^2_2 = p^T R^T R p = p^T p.$$

5 The Lie Algebra $so(3)$

The Lie algebra of SO(3) is

$$\boxed{\mathfrak{so}(3) = \left\{\Omega \in \mathbb{R}^{3\times 3} : \Omega^T = -\Omega\right\}.}$$

Thus $\mathfrak{so}(3)$ is the vector space of skew-symmetric 3×3 matrices.

A general element is

$$\Omega = \begin{bmatrix} 0 & -\omega_z & \omega_y \\ \omega_z & 0 & -\omega_x \\ -\omega_y & \omega_x & 0 \end{bmatrix}.$$

5.1 Hat Operator

For

$$\omega = \begin{bmatrix} \omega_x \\ \omega_y \\ \omega_z \end{bmatrix} \in \mathbb{R}^3,$$

define

$$\boxed{\omega^\wedge = \begin{bmatrix} 0 & -\omega_z & \omega_y \\ \omega_z & 0 & -\omega_x \\ -\omega_y & \omega_x & 0 \end{bmatrix}.}$$

The mapping

$$\wedge : \mathbb{R}^3 \to \mathfrak{so}(3)$$

is called the hat operator.

5.2 Vee Operator

The inverse mapping

$$\vee : \mathfrak{so}(3) \to \mathbb{R}^3$$

is called the vee operator.
For

$$\Omega = \begin{bmatrix} 0 & -\omega_z & \omega_y \\ \omega_z & 0 & -\omega_x \\ -\omega_y & \omega_x & 0 \end{bmatrix},$$

we have

$$\boxed{\Omega^\vee = \begin{bmatrix} \omega_x \\ \omega_y \\ \omega_z \end{bmatrix}.}$$

Therefore

$$(\omega^\wedge)^\vee = \omega.$$

5.3 Cross Product

The hat operator converts the cross product into matrix multiplication:

$$\boxed{\omega^\wedge v = \omega \times v.}$$

Explicitly,

$$\begin{bmatrix} 0 & -\omega_z & \omega_y \\ \omega_z & 0 & -\omega_x \\ -\omega_y & \omega_x & 0 \end{bmatrix} \begin{bmatrix} v_x \\ v_y \\ v_z \end{bmatrix} = \omega \times v.$$

6 Lie Bracket

For matrices A, B, the Lie bracket is

$$\boxed{[A, B] = AB - BA.}$$

For

$$\Omega_1 = \omega_1^\wedge, \qquad \Omega_2 = \omega_2^\wedge,$$

we obtain

$$[\omega_1^\wedge, \omega_2^\wedge] = (\omega_1 \times \omega_2)^\wedge.$$

Thus the Lie bracket on $\mathfrak{so}(3)$ corresponds to the cross product:

$$\boxed{[\omega_1, \omega_2] = \omega_1 \times \omega_2.}$$

The Lie bracket measures the noncommutativity of infinitesimal transformations.

7 Exponential Map on $SO(3)$

The matrix exponential is

$$\exp(A) = \sum_{k=0}^{\infty} \frac{A^k}{k!}.$$

For

$$\omega^\wedge \in \mathfrak{so}(3),$$

the exponential map produces a rotation:

$$\boxed{R = \exp\bigl(\omega^\wedge\bigr).}$$

Therefore

$$\exp : \mathfrak{so}(3) \to \mathrm{SO}(3).$$

7.1 Axis-Angle Representation

Let

$$\theta = \|\omega\|_{2\,2}.$$

If $\theta \neq 0$, define

$$u = \frac{\omega}{\theta}.$$

Then

$$\omega = \theta u,$$

where

$$\|u\|_{2\,2} = 1.$$

The vector u is the rotation axis and θ is the rotation angle.

7.2 Rodrigues' Formula

For

$$R = \exp(\omega^\wedge),$$

Rodrigues' formula gives

$$\boxed{R = I + \frac{\sin\theta}{\theta}\omega^\wedge + \frac{1-\cos\theta}{\theta^2}(\omega^\wedge)^2.}$$

Equivalently,

$$R = I + \sin\theta\, u^\wedge + (1-\cos\theta)(u^\wedge)^2.$$

7.3 Small-Angle Expansion

For small θ,

$$\sin\theta = \theta - \frac{\theta^3}{6} + O(\theta^5),$$

and

$$1-\cos\theta = \frac{\theta^2}{2} - \frac{\theta^4}{24} + O(\theta^6).$$

Hence

$$\frac{\sin\theta}{\theta} \approx 1 - \frac{\theta^2}{6},$$

and

$$\frac{1-\cos\theta}{\theta^2} \approx \frac{1}{2} - \frac{\theta^2}{24}.$$

These expansions are useful for numerical implementations near $\theta = 0$.

8 Logarithm Map on $SO(3)$

The logarithm map is locally the inverse of the exponential map:

$$\boxed{\log : \mathrm{SO}(3) \to \mathfrak{so}(3).}$$

Given

$$R \in \mathrm{SO}(3),$$

we seek

$$\omega^\wedge = \log(R).$$

The corresponding vector is

$$\boxed{\omega = \log(R)^\vee.}$$

8.1 Rotation Angle

The rotation angle is

$$\boxed{\theta = \cos^{-1}\left(\frac{\mathrm{tr}(R)-1}{2}\right).}$$

For

$$0 < \theta < \pi,$$

the logarithm is

$$\boxed{\log(R) = \frac{\theta}{2\sin\theta}(R - R^T).}$$

Therefore

$$\boxed{\log(R)^\vee = \frac{\theta}{2\sin\theta}(R - R^T)^\vee.}$$

8.2 Relative Rotation

For two orientations

$$R_1, R_2 \in \mathrm{SO}(3),$$

the relative rotation is

$$\boxed{R_{\mathrm{rel}} = R_1^T R_2.}$$

Its logarithmic representation is

$$\boxed{\phi_{\mathrm{rel}} = \log\big(R_1^T R_2\big)^\vee.}$$

This provides a three-dimensional local representation of the relative orientation.

9 Rotation Kinematics

Let

$$R(t) \in \mathrm{SO}(3)$$

describe the orientation of a rigid body.
Differentiating

$$R^T R = I$$

gives

$$\dot{R}^T R + R^T \dot{R} = 0.$$

Therefore

$$R^T \dot{R}$$

is skew-symmetric.

Hence there exists

$$\omega \in \mathbb{R}^3$$

such that

$$\boxed{R^T \dot{R} = \omega^\wedge.}$$

Therefore

$$\boxed{\dot{R} = R\omega^\wedge.}$$

This is the body-frame angular velocity representation.
Alternatively,

$$\boxed{\dot{R} = \omega_s^\wedge R}$$

represents spatial-frame angular velocity.

10 The Special Euclidean Group $SE(3)$

A rigid-body pose consists of a rotation and a translation.
It is represented by

$$\boxed{T = \begin{bmatrix} R & p \\ 0 & 1 \end{bmatrix},}$$

where

$$R \in \mathrm{SO}(3), \qquad p \in \mathbb{R}^3.$$

The set of all such matrices is

$$\boxed{\mathrm{SE}(3) = \left\{ \begin{bmatrix} R & p \\ 0 & 1 \end{bmatrix} : R \in \mathrm{SO}(3),\ p \in \mathbb{R}^3 \right\}.}$$

10.1 Homogeneous Coordinates

Represent a point as

$$\bar{p} = \begin{bmatrix} p \\ 1 \end{bmatrix}.$$

Then

$$\bar{p}' = T\bar{p}.$$

Therefore

$$\begin{bmatrix} p' \\ 1 \end{bmatrix} = \begin{bmatrix} R & p \\ 0 & 1 \end{bmatrix} \begin{bmatrix} p \\ 1 \end{bmatrix}.$$

Thus

$$\boxed{p' = Rp + p.}$$

More precisely, if the translation is denoted t,

$$\boxed{p' = Rp + t.}$$

10.2 Composition

Let

$$T_1 = \begin{bmatrix} R_1 & p_1 \\ 0 & 1 \end{bmatrix},$$

and

$$T_2 = \begin{bmatrix} R_2 & p_2 \\ 0 & 1 \end{bmatrix}.$$

Then

$$T_1 T_2 = \begin{bmatrix} R_1 R_2 & R_1 p_2 + p_1 \\ 0 & 1 \end{bmatrix}.$$

Thus

$$\boxed{R_{12} = R_1 R_2}$$

and

$$\boxed{p_{12} = R_1 p_2 + p_1.}$$

10.3 Inverse

For

$$T = \begin{bmatrix} R & p \\ 0 & 1 \end{bmatrix},$$

the inverse is

$$\boxed{T^{-1} = \begin{bmatrix} R^T & -R^T p \\ 0 & 1 \end{bmatrix}.}$$

10.4 Degrees of Freedom

The group SE(3) has six degrees of freedom:

$$3$$

for translation and

$$3$$

for rotation.
Therefore

$$\boxed{\dim \mathrm{SE}(3) = 6.}$$

11 The Lie Algebra $se(3)$

The Lie algebra of SE(3) is

$$\boxed{\mathfrak{se}(3) = \left\{ \begin{bmatrix} \omega^\wedge & v \\ 0 & 0 \end{bmatrix} : v, \omega \in \mathbb{R}^3 \right\}.}$$

An element of $\mathfrak{se}(3)$ is associated with a six-dimensional vector

$$\boxed{\xi = \begin{bmatrix} v \\ \omega \end{bmatrix} \in \mathbb{R}^6.}$$

The vector ξ is called a twist coordinate vector.

11.1 Hat Operator for $se(3)$

Define

$$\boxed{\xi^\wedge = \begin{bmatrix} \omega^\wedge & v \\ 0 & 0 \end{bmatrix}.}$$

Explicitly,

$$\xi^\wedge = \begin{bmatrix} 0 & -\omega_z & \omega_y & v_x \\ \omega_z & 0 & -\omega_x & v_y \\ -\omega_y & \omega_x & 0 & v_z \\ 0 & 0 & 0 & 0 \end{bmatrix}.$$

The vee operator gives

$$\boxed{(\xi^\wedge)^\vee = \begin{bmatrix} v \\ \omega \end{bmatrix}.}$$

12 Twists

A twist represents rigid-body velocity.

Define

$$\xi = \begin{bmatrix} v \\ \omega \end{bmatrix}.$$

Here

$$v$$

is linear velocity and

$$\omega$$

is angular velocity.

The corresponding matrix is

$$\xi^\wedge = \begin{bmatrix} \omega^\wedge & v \\ 0 & 0 \end{bmatrix}.$$

12.1 Rigid-Body Velocity

Let

$$T(t) \in \mathrm{SE}(3).$$

The body twist is defined by

$$\boxed{\xi_b^\wedge = T^{-1}\dot{T}.}$$

Therefore

$$\boxed{\dot{T} = T\xi_b^\wedge.}$$

The spatial twist is

$$\boxed{\xi_s^\wedge = \dot{T}T^{-1}.}$$

Therefore

$$\boxed{\dot{T} = \xi_s^\wedge T.}$$

13 Exponential Map on $SE(3)$

The exponential map is

$$\boxed{\exp : \mathfrak{se}(3) \to \mathrm{SE}(3).}$$

For

$$\xi = \begin{bmatrix} v \\ \omega \end{bmatrix},$$

we have

$$\exp\big(\xi^\wedge\big) = \begin{bmatrix} R & p \\ 0 & 1 \end{bmatrix}.$$

The rotation part is

$$R = \exp\big(\omega^\wedge\big).$$

The translation part is

$$p = J(\omega)v,$$

where $J(\omega)$ is the left Jacobian of SO(3).
Therefore

$$\boxed{\exp\big(\xi^\wedge\big) = \begin{bmatrix} \exp(\omega^\wedge) & J(\omega)v \\ 0 & 1 \end{bmatrix}.}$$

13.1 The Left Jacobian of $SO(3)$

Let

$$\theta = \|\omega\|_{2\,2}.$$

Then

$$\boxed{J(\omega) = I + \frac{1-\cos\theta}{\theta^2}\omega^\wedge + \frac{\theta-\sin\theta}{\theta^3}(\omega^\wedge)^2.}$$

Thus

$$p = J(\omega)v.$$

13.2 Small-Angle Jacobian

For small θ,

$$\boxed{J(\omega) \approx I + \frac{1}{2}\omega^\wedge + \frac{1}{6}(\omega^\wedge)^2.}$$

14 Logarithm Map on $SE(3)$

Given

$$T = \begin{bmatrix} R & p \\ 0 & 1 \end{bmatrix},$$

the logarithm has the form

$$\boxed{\log(T) = \begin{bmatrix} \omega^\wedge & v \\ 0 & 0 \end{bmatrix}.}$$

The rotational component is

$$\omega = \log(R)^\vee.$$

The translational component is obtained from

$$\boxed{v = J^{-1}(\omega)p.}$$

Therefore

$$\boxed{\log(T)^\vee = \begin{bmatrix} J^{-1}(\omega)p \\ \omega \end{bmatrix}.}$$

15 Adjoint Representation

Let

$$T = \begin{bmatrix} R & p \\ 0 & 1 \end{bmatrix}.$$

The adjoint representation of T is the 6×6 matrix

$$\boxed{\mathrm{Ad}_T = \begin{bmatrix} R & p^\wedge R \\ 0 & R \end{bmatrix}.}$$

For a body twist

$$\xi_b = \begin{bmatrix} v_b \\ \omega_b \end{bmatrix},$$

the corresponding spatial twist is

$$\boxed{\xi_s = \mathrm{Ad}_T\, \xi_b.}$$

Explicitly,

$$v_s = Rv_b + p^\wedge R\omega_b,$$

and

$$\omega_s = R\omega_b.$$

15.1 Frame Transformation

Suppose

$$T_{AB}$$

maps coordinates from frame B to frame A.
Then a twist expressed in frame B is transformed into frame A using

$$\boxed{\xi_A = \mathrm{Ad}_{T_{AB}}\, \xi_B.}$$

This is central to robotic kinematics.

16 The Adjoint Lie Algebra

For

$$\xi = \begin{bmatrix} v \\ \omega \end{bmatrix},$$

define

$$\boxed{\mathrm{ad}_\xi = \begin{bmatrix} \omega^\wedge & v^\wedge \\ 0 & \omega^\wedge \end{bmatrix}.}$$

The Lie bracket satisfies

$$[\xi_1^\wedge, \xi_2^\wedge] = [\xi_1, \xi_2]^\wedge.$$

For

$$\xi_1 = \begin{bmatrix} v_1 \\ \omega_1 \end{bmatrix}, \qquad \xi_2 = \begin{bmatrix} v_2 \\ \omega_2 \end{bmatrix},$$

we obtain

$$\boxed{[\xi_1, \xi_2] = \begin{bmatrix} \omega_1 \times v_2 - \omega_2 \times v_1 \\ \omega_1 \times \omega_2 \end{bmatrix}.}$$

17 Baker–Campbell–Hausdorff Formula

For two Lie-algebra elements A and B,

$$\log\bigl(e^A e^B\bigr)$$

is generally not equal to

$$A + B.$$

The Baker–Campbell–Hausdorff expansion is

$$\boxed{\log\bigl(e^A e^B\bigr) = A + B + \frac{1}{2}[A, B] + \frac{1}{12}[A, [A, B]] + \frac{1}{12}[B, [B, A]] + \cdots}$$

The first-order approximation is

$$\log\bigl(e^A e^B\bigr) \approx A + B$$

when A and B are sufficiently small.
This explains why finite rotations cannot generally be combined by ordinary vector addition.

18 Left and Right Perturbations

A perturbation of

$$T \in \mathrm{SE}(3)$$

can be applied on either side.
A right perturbation is

$$\boxed{T' = T \exp\bigl(\delta\xi^\wedge\bigr).}$$

A left perturbation is

$$\boxed{T' = \exp\bigl(\delta\xi^\wedge\bigr) T.}$$

Right perturbations are naturally associated with local/body-frame coordinates.
Left perturbations are naturally associated with global/spatial coordinates.

19 Geodesic Distance on $SO(3)$

For

$$R_1, R_2 \in \mathrm{SO}(3),$$

define

$$R_{\mathrm{err}} = R_1^T R_2.$$

The intrinsic rotation distance is

$$\boxed{d(R_1, R_2) = \left\| \log\bigl(R_1^T R_2\bigr)^\vee \right\|_2 .}$$

Since

$$\log\left(R_1^T R_2\right)^\vee$$

is the axis-angle vector, this distance is the magnitude of the smallest rotation connecting the two orientations.

20 Relative Pose Error on $SE(3)$

Let

$$T_d$$

be a desired pose and

$$T$$

be the current pose.
Define the relative transformation

$$\boxed{T_e = T_d^{-1} T.}$$

The Lie-algebra error is

$$\boxed{e_T = \log\left(T_d^{-1} T\right)^\vee.}$$

Thus

$$e_T \in \mathbb{R}^6.$$

Write

$$e_T = \begin{bmatrix} e_p \\ e_R \end{bmatrix}.$$

Then

$$e_p \in \mathbb{R}^3$$

is the translational component and

$$e_R \in \mathbb{R}^3$$

is the rotational component.

21 Robot Kinematics

Let a robot have configuration

$$q = \begin{bmatrix} q_1 & \cdots & q_n \end{bmatrix}^T.$$

The end-effector pose is

$$T(q) \in \mathrm{SE}(3).$$

Forward kinematics defines

$$\boxed{T = T(q).}$$

21.1 Product of Exponentials

For a serial manipulator,

$$\boxed{T(q) = e^{\xi_1^\wedge q_1} e^{\xi_2^\wedge q_2} \cdots e^{\xi_n^\wedge q_n} M,}$$

where

$$M$$

is the home configuration and

$$\xi_i$$

is the screw axis associated with joint i.

21.2 Manipulator Jacobian

The end-effector twist satisfies

$$\boxed{V = J(q)\dot{q}.}$$

The twist coordinates are

$$V = \begin{bmatrix} v \\ \omega \end{bmatrix}.$$

Therefore

$$\boxed{\xi = J(q)\dot{q}.}$$

The Jacobian provides the local mapping

$$\mathbb{R}^n \to \mathfrak{se}(3).$$

22 Configuration Spaces of Robots

A robot configuration may have the form

$$q \in \mathcal{Q}.$$

For a revolute joint,

$$q_i \in S^1.$$

For a prismatic joint,

$$q_i \in \mathbb{R}.$$

For a free rigid body,

$$q \in \mathrm{SE}(3).$$

A general robot configuration space can therefore have the product structure

$$\mathcal{Q} = S^1 \times \cdots \times S^1 \times \mathbb{R}^m \times \mathrm{SE}(3).$$

23 Lie Groups in Reinforcement Learning

A standard reinforcement-learning problem is described by a Markov decision process

$$(\mathcal{S}, \mathcal{A}, P, r, \gamma).$$

A state satisfies

$$s_t \in \mathcal{S},$$

and an action satisfies

$$a_t \in \mathcal{A}.$$

The policy is

$$a_t \sim \pi_\theta(\cdot|s_t).$$

In robotics, parts of $\mathcal{S}$ or $\mathcal{A}$ may naturally belong to manifolds.
For example,

$$R_t \in \mathrm{SO}(3)$$

or

$$T_t \in \mathrm{SE}(3).$$

23.1 Manifold-Valued State

Consider

$$s_t = (T_t, \dot{q}_t, x_t, \ldots),$$

where

$$T_t \in \mathrm{SE}(3).$$

The state is not globally represented by a vector in a single Euclidean space.
A local representation relative to a desired pose is

$$\boxed{e_t = \log\big(T_d^{-1} T_t\big)^\vee.}$$

Then

$$e_t \in \mathbb{R}^6.$$

A learning system can therefore use

$$s_t^{\mathrm{RL}} = \begin{bmatrix} e_t \\ \dot{q}_t \\ x_t \end{bmatrix}.$$

24 Lie-Algebra Action Spaces

Suppose the RL policy produces

$$\xi_t = \pi_\theta(s_t) \in \mathbb{R}^6.$$

Interpret

$$\xi_t = \begin{bmatrix} v_t \\ \omega_t \end{bmatrix}$$

as an element of $se(3)$.
Construct

$$\xi_t^\wedge = \begin{bmatrix} \omega_t^\wedge & v_t \\ 0 & 0 \end{bmatrix}.$$

The pose update is

$$\boxed{T_{t+1} = T_t \exp\big(\alpha \xi_t^\wedge\big),}$$

where $\alpha > 0$ is the integration step.
Because

$$\exp\big(\alpha \xi_t^\wedge\big) \in \mathrm{SE}(3),$$

the updated pose remains in

$$\mathrm{SE}(3).$$

25 Orientation Actions

For orientation-only control, a policy can produce

$$\delta\theta_t \in \mathbb{R}^3.$$

The update is

$$\boxed{R_{t+1} = R_t \exp\big(\delta\theta_t^\wedge\big).}$$

This guarantees

$$R_{t+1} \in \mathrm{SO}(3).$$

26 Geometric State Representations

Suppose the current pose is

$$T_t$$

and the target pose is

$$T_d.$$

Define

$$T_e = T_d^{-1} T_t.$$

Then

$$e_t = \log(T_e)^\vee.$$

The RL observation can contain

$$\boxed{s_t = \begin{bmatrix} e_t \\ \dot{q}_t \\ \dot{x}_t \\ \text{sensor features} \end{bmatrix}.}$$

This representation encodes the relative pose rather than the absolute matrix entries.

27 Geometric Reward Functions

A pose-tracking reward can be constructed from the Lie-algebra error.

Let

$$e_T = \log\bigl(T_d^{-1} T\bigr)^\vee.$$

A quadratic reward is

$$\boxed{r_t = -e_T^T Q e_T - u_t^T R u_t,}$$

where

$$Q \succeq 0, \qquad R \succeq 0.$$

Separating translation and rotation gives

$$e_T = \begin{bmatrix} e_p \\ e_R \end{bmatrix}.$$

Then

$$\boxed{r_t = -w_p \left\|e_p\right\|_2{}_2^2 - w_R \left\|e_R\right\|_2{}_2^2 - w_u \left\|u_t\right\|_2{}_2^2.}$$

28 Geometric RL Example

Consider an end-effector that must reach

$$T_d.$$

At time t:

$$T_t \in \mathrm{SE}(3).$$

Calculate

$$T_{e,t} = T_d^{-1} T_t.$$

Then

$$e_t = \log(T_{e,t})^\vee.$$

The policy receives

$$s_t = \begin{bmatrix} e_t \\ \dot{q}_t \end{bmatrix}.$$

The policy produces

$$\xi_t = \pi_\theta(s_t).$$

The environment performs

$$T_{t+1} = T_t \exp\bigl(\alpha \xi_t^\wedge\bigr).$$

The reward is

$$r_t = -w_e \left\|e_t\right\|_2 {}_2^2 - w_\xi \left\|\xi_t\right\|_2 {}_2^2.$$

The complete RL loop is therefore

$$\boxed{T_t \to \log\bigl(T_d^{-1} T_t\bigr) \to s_t \to \pi_\theta \to \xi_t \to \exp\bigl(\xi_t^\wedge\bigr) \to T_{t+1}.}$$

29 Lie Groups and Policy Learning

Let

$$\pi_\theta : \mathcal{S} \to \mathbb{R}^6$$

be a neural policy.
The output

$$\pi_\theta(s) = \xi$$

is interpreted as an element of the tangent space

$$se(3).$$

The environment maps this tangent vector into the configuration space:

$$\boxed{\xi \xrightarrow{\exp} T_{\text{increment}} \in \mathrm{SE}(3).}$$

The resulting architecture is

$$\boxed{\text{state} \to \text{policy} \to \mathfrak{se}(3) \to \exp \to \mathrm{SE}(3).}$$

30 Lie Groups in Model-Based RL

Suppose a learned dynamics model predicts a local motion:

$$\hat{\xi}_t = f_\theta(s_t, a_t).$$

Instead of directly predicting the next pose, integrate the predicted Lie-algebra velocity:

$$\boxed{T_{t+1} = T_t \exp\Bigl(\Delta t\, \hat{\xi}_t^\wedge\Bigr).}$$

This gives a geometric dynamics model

$$f_\theta : \mathcal{S} \times \mathcal{A} \to \mathfrak{se}(3).$$

The group update is then performed analytically.

31 Lie Groups in Imitation Learning

Suppose demonstrations contain

$$T_0, T_1, \dots, T_N.$$

The local motion between consecutive demonstrations is

$$\boxed{\xi_t = \frac{1}{\Delta t} \log\big(T_t^{-1} T_{t+1}\big)^\vee.}$$

A policy can be trained to predict

$$\hat{\xi}_t = \pi_\theta(s_t).$$

The imitation-learning loss can be

$$\boxed{\mathcal{L} = \sum_t \left\| \hat{\xi}_t - \xi_t \right\|_2^{\,2}.}$$

32 Lie Groups in Trajectory Optimization

A trajectory consists of

$$T_0, T_1, \dots, T_N.$$

A geometric trajectory cost can use

$$e_k = \log\big(T_d^{-1} T_k\big)^\vee.$$

Then

$$\boxed{J = \sum_{k=0}^{N} e_k^T Q e_k + \sum_{k=0}^{N-1} u_k^T R u_k.}$$

For relative trajectory smoothness, use

$$\delta\xi_k = \log\big(T_k^{-1} T_{k+1}\big)^\vee.$$

Then

$$J_{\text{smooth}} = \sum_{k=0}^{N-1} \|\delta\xi_k\|_2^{\,2}.$$

33 Geometric Control Error

For desired pose

$$T_d$$

and current pose

$$T,$$

define

$$T_e = T_d^{-1} T.$$

Then

$$e_T = \log(T_e)^\vee.$$

A geometric controller may use

$$\boxed{u = -K e_T.}$$

For

$$K = \begin{bmatrix} K_p & 0 \\ 0 & K_R \end{bmatrix},$$

the control law becomes

$$u = -\begin{bmatrix} K_p & 0 \\ 0 & K_R \end{bmatrix} \begin{bmatrix} e_p \\ e_R \end{bmatrix}.$$

An RL policy can learn to replace or augment such a controller.

34 Body and Spatial Coordinates

Let

$$T_{AB}$$

denote the transformation from frame B to frame A.
A body twist is represented in the moving body frame:

$$\xi_B.$$

A spatial twist is represented in the reference frame:

$$\xi_A.$$

They satisfy

$$\boxed{\xi_A = \mathrm{Ad}_{T_{AB}} \xi_B.}$$

Consequently, an RL policy must maintain consistency between the frame in which an action is generated and the frame in which it is applied.

35 Invariance

Let a group G act on a space $\mathcal{X}$.

A function

$$f : \mathcal{X} \to \mathcal{Y}$$

is invariant under the group action if

$$\boxed{f(g \cdot x) = f(x)}$$

for all

$$g \in G.$$

For robotic tasks, quantities such as relative pose errors can be constructed to avoid dependence on an arbitrary global coordinate frame.

For example,

$$e = \log\left(T_d^{-1} T\right)^{\vee}$$

depends on the relative transformation between the current and target poses.

36 Equivariance

A function

$$f : \mathcal{X} \to \mathcal{Y}$$

is equivariant if

$$\boxed{f(g \cdot x) = g \cdot f(x).}$$

In robotics, equivariance expresses consistent transformation of outputs when the physical problem is transformed.

For a policy,

$$\pi : \mathcal{S} \to \mathcal{A},$$

an equivariance condition can be written

$$\boxed{\pi(g \cdot s) = g \cdot \pi(s).}$$

The exact group action depends on the chosen state and action representations.

37 $SE(3)$-Equivariant Reinforcement Learning

For rigid-body robotics, the relevant symmetry group is often

$$SE(3).$$

A geometric policy can be constructed so that its outputs transform consistently under rigid-body transformations.

The conceptual structure is

$$s \xrightarrow{g} g \cdot s$$

and

$$\pi(g \cdot s) = g \cdot \pi(s).$$

Such constructions are useful for:

- coordinate-frame consistency,
- generalization across orientations,
- geometric representation learning,
- sample-efficient policy learning,
- manipulation,
- locomotion,
- aerial robotics.

38 Orientation Representations in RL

Several representations are commonly used.

38.1 Euler Angles

An orientation can be parameterized by

$$(\phi, \theta, \psi).$$

For example,

$$R = R_z(\psi) R_y(\theta) R_x(\phi).$$

Euler angles can have singularities and discontinuities.

38.2 Rotation Matrices

A rotation matrix contains

$$9$$

numbers subject to constraints:

$$R^T R = I, \qquad \det R = 1.$$

38.3 Quaternions

A unit quaternion can be written

$$q = \begin{bmatrix} q_0 \\ q_1 \\ q_2 \\ q_3 \end{bmatrix}, \qquad \|q\|_{2\,2} = 1.$$

It provides a compact orientation representation.

38.4 Lie-Algebra Coordinates

A local rotation can be represented by

$$\phi = \log(R)^\vee \in \mathbb{R}^3.$$

Relative orientation becomes

$$\boxed{\phi_e = \log\big(R_d^T R\big)^\vee.}$$

This is particularly convenient for local errors and incremental actions.

39 Numerical Implementation

Numerical implementations must handle special cases carefully.

39.1 Small Rotations

When

$$\theta \approx 0,$$

expressions such as

$$\frac{\sin\theta}{\theta}$$

should be evaluated using series expansions.
For example,

$$\frac{\sin\theta}{\theta} \approx 1 - \frac{\theta^2}{6} + \frac{\theta^4}{120}.$$

Similarly,

$$\frac{1-\cos\theta}{\theta^2} \approx \frac{1}{2} - \frac{\theta^2}{24} + \frac{\theta^4}{720}.$$

39.2 Rotations Near π

The logarithm map becomes numerically delicate near

$$\theta = \pi.$$

The axis-angle representation is not unique at π because

$$(\pi, u)$$

and

$$(-\pi, -u)$$

represent the same rotation.
Implementations of $\log(R)$ therefore require special handling near π.

39.3 Projection onto $SO(3)$

If a numerical procedure produces an approximately orthogonal matrix A, an $SO(3)$ projection can be obtained from the singular value decomposition

$$A = U\Sigma V^T.$$

A projected rotation is

$$R = UV^T.$$

If

$$\det\bigl(UV^T\bigr) < 0,$$

one singular-vector sign must be adjusted to ensure

$$\det R = 1.$$

40 PyTorch Implementation

A basic hat operator can be implemented as follows:

```
import torch

def hat(w):
    wx = w[..., 0]
    wy = w[..., 1]
    wz = w[..., 2]

    O = torch.zeros_like(wx)

    return torch.stack([
        torch.stack([O, -wz, wy], dim=-1),
        torch.stack([wz, O, -wx], dim=-1),
        torch.stack([-wy, wx, O], dim=-1)
    ], dim=-2)
```

For an RL policy,

```
xi = policy(state)
```

where

```
xi[..., :3]
```

contains the translational component and

```
xi[..., 3:]
```

contains the rotational component.

The corresponding $se(3)$ matrix is

```
def se3_hat(xi):
    v = xi[..., :3]
    w = xi[..., 3:]

    W = hat(w)

    T = torch.zeros(
```

```
        *xi.shape[:-1], 4, 4,
        device=xi.device,
        dtype=xi.dtype
    )

    T[..., :3, :3] = W
    T[..., :3, 3] = v

    return T
```

41 Geometric RL Environment

A geometric RL environment can use the following structure:

```
class GeometricRobotEnv:

    def reset(self):
        T = initial_pose()
        return observation(T)

    def observation(self, T):
        T_error = inverse(T_goal) @ T
        error = se3_log(T_error)
        return error

    def step(self, action):

        xi = action

        T_increment = se3_exp(xi)

        T = T @ T_increment

        reward = pose_reward(T)

        observation = self.observation(T)

        return observation, reward
```

The mathematical transition is

$$\boxed{T_{t+1} = T_t \exp\bigl(\xi_t^{\wedge}\bigr).}$$

The observation is

$$\boxed{s_t = \log\bigl(T_d^{-1} T_t\bigr)^{\vee}.}$$

42 Reinforcement Learning Algorithms

Lie-group representations are compatible with standard RL algorithms.

For example, a Soft Actor-Critic policy can be written as

$$\pi_\theta(a|s),$$

with

$$a = \xi \in \mathbb{R}^6.$$

The action is interpreted geometrically through

$$T_{t+1} = T_t \exp\bigl(\xi^\wedge\bigr).$$

Similarly, PPO can use

$$\xi_t = \pi_\theta(s_t)$$

followed by a Lie-group state update.
The RL algorithm and geometric representation therefore occupy different layers:

$$\boxed{\text{RL algorithm} + \text{geometric state/action representation.}}$$

43 Example: SAC with an $SE(3)$ Action

Let

$$s_t = \begin{bmatrix} \log\bigl(T_d^{-1} T_t\bigr)^\vee \\ \dot{q}_t \end{bmatrix}.$$

The actor produces

$$\xi_t \sim \pi_\theta(\cdot|s_t).$$

The environment applies

$$T_{t+1} = T_t \exp\bigl(\Delta t\, \xi_t^\wedge\bigr).$$

The reward is

$$r_t = -w_p \left\|e_{p,t}\right\|_2{}_2^2 - w_R \left\|e_{R,t}\right\|_2{}_2^2 - w_\xi \left\|\xi_t\right\|_2{}_2^2.$$

The critic estimates

$$Q_\phi(s_t, \xi_t).$$

Thus the complete system is

$$\boxed{\begin{aligned} T_t &\to e_t \\ &\to s_t \\ &\to \pi_\theta \\ &\to \xi_t \\ &\to \exp\bigl(\xi_t^\wedge\bigr) \\ &\to T_{t+1}. \end{aligned}}$$

44 Example: PPO with Orientation Control

Suppose

$$R_t \in \mathrm{SO}(3).$$

The target is

$$R_d.$$

Define

$$e_t = \log\bigl(R_d^T R_t\bigr)^\vee.$$

The state is

$$s_t = \begin{bmatrix} e_t \\ \omega_t \end{bmatrix}.$$

The PPO actor outputs

$$\delta\theta_t \in \mathbb{R}^3.$$

The orientation update is

$$\boxed{R_{t+1} = R_t \exp\bigl(\delta\theta_t^\wedge\bigr).}$$

The reward is

$$\boxed{r_t = -\alpha \left\|e_t\right\|_2{}_2^2 - \beta \left\|\delta\theta_t\right\|_2{}_2^2.}$$

45 Quadrotor Example

A quadrotor state contains

$$p \in \mathbb{R}^3,$$

$$v \in \mathbb{R}^3,$$

$$R \in \mathrm{SO}(3),$$

and

$$\omega \in \mathbb{R}^3.$$

A geometric state representation relative to a target is

$$\boxed{s = \begin{bmatrix} p - p_d \\ \log\bigl(R_d^T R\bigr)^\vee \\ v - v_d \\ \omega - \omega_d \end{bmatrix}.}$$

A policy can output control variables such as

$$u = \begin{bmatrix} f \\ \tau \end{bmatrix},$$

while the Lie-group structure describes the attitude.

46 Manipulator Example

For a manipulator,

$$q \in \mathbb{R}^n.$$

Forward kinematics gives

$$T(q) \in \mathrm{SE}(3).$$

The target pose is

$$T_d.$$

The geometric error is

$$e_T = \log\big(T_d^{-1} T(q)\big)^\vee.$$

A policy can receive

$$s = \begin{bmatrix} e_T \\ \dot{q} \end{bmatrix}.$$

If the policy outputs a Cartesian twist

$$\xi,$$

then

$$\xi = J(q)\dot{q}.$$

If $J(q)$ has full row rank, one possible joint velocity is

$$\boxed{\dot{q} = J(q)^\dagger \xi.}$$

The RL policy can therefore operate in Cartesian Lie-algebra coordinates while the robot executes joint-space commands.

47 Geometric Error versus Euclidean Error

For ordinary vectors,

$$e = x_d - x.$$

For rotations, direct subtraction

$$R_d - R$$

does not provide an intrinsic rotation error.
Instead,

$$\boxed{e_R = \log\big(R_d^T R\big)^\vee.}$$

For poses,

$$\boxed{e_T = \log\big(T_d^{-1} T\big)^\vee.}$$

Thus the group operation constructs the relative configuration and the logarithm converts it into a local vector.

48 Important Identities

For

$$R \in \mathrm{SO}(3),$$

we have

$$R^{-1} = R^T.$$

For

$$\omega \in \mathbb{R}^3,$$

$$\omega^\wedge v = \omega \times v.$$

For

$$R = \exp\bigl(\omega^\wedge\bigr),$$

$$R \in \mathrm{SO}(3).$$

For

$$T = \exp\bigl(\xi^\wedge\bigr),$$

$$T \in \mathrm{SE}(3).$$

For

$$T = \begin{bmatrix} R & p \\ 0 & 1 \end{bmatrix},$$

$$T^{-1} = \begin{bmatrix} R^T & -R^T p \\ 0 & 1 \end{bmatrix}.$$

For relative pose,

$$T_e = T_d^{-1} T.$$

For geometric pose error,

$$e_T = \log(T_e)^\vee.$$

For body-to-spatial twist conversion,

$$\xi_s = \mathrm{Ad}_T\, \xi_b.$$

49 Summary of the Mathematical Structure

The central relationships are

$$\boxed{\mathrm{SO}(3) \longleftrightarrow \mathfrak{so}(3)}$$

and

$$\boxed{\mathrm{SE}(3) \longleftrightarrow \mathfrak{se}(3).}$$

The corresponding exponential maps are

$$\boxed{\exp : \mathfrak{so}(3) \to \mathrm{SO}(3)}$$

and

$$\boxed{\exp : \mathfrak{se}(3) \to \mathrm{SE}(3).}$$

The logarithm maps provide local coordinates:

$$\boxed{\log : \mathrm{SO}(3) \to \mathfrak{so}(3)}$$

and

$$\boxed{\log : \mathrm{SE}(3) \to \mathfrak{se}(3).}$$

The hat and vee operators establish

$$\mathbb{R}^3 \leftrightarrow \mathfrak{so}(3)$$

and

$$\mathbb{R}^6 \leftrightarrow \mathfrak{se}(3).$$

The adjoint representation provides frame transformations:

$$\boxed{\xi_A = \mathrm{Ad}_{T_{AB}}\, \xi_B.}$$

50 Core Formula Sheet

$SO(3)$

$$\boxed{SO(3) = \{R : R^T R = I, \det R = 1\}}$$

$$\boxed{\omega^\wedge = \begin{bmatrix} 0 & -\omega_z & \omega_y \\ \omega_z & 0 & -\omega_x \\ -\omega_y & \omega_x & 0 \end{bmatrix}}$$

$$\boxed{\omega^\wedge v = \omega \times v}$$

$$\boxed{R = \exp\left(\omega^\wedge\right)}$$

$$\boxed{\exp\left(\omega^\wedge\right) = I + \frac{\sin\theta}{\theta}\omega^\wedge + \frac{1-\cos\theta}{\theta^2}(\omega^\wedge)^2}$$

$$\boxed{\theta = \cos^{-1}\left(\frac{\mathrm{tr}(R)-1}{2}\right)}$$

$$\boxed{\log(R) = \frac{\theta}{2\sin\theta}(R - R^T)}$$

$$\boxed{e_R = \log\left(R_d^T R\right)^\vee}$$

$SE(3)$

$$\boxed{T = \begin{bmatrix} R & p \\ 0 & 1 \end{bmatrix}}$$

$$\boxed{T^{-1} = \begin{bmatrix} R^T & -R^T p \\ 0 & 1 \end{bmatrix}}$$

$$\boxed{\xi = \begin{bmatrix} v \\ \omega \end{bmatrix}}$$

$$\boxed{\xi^\wedge = \begin{bmatrix} \omega^\wedge & v \\ 0 & 0 \end{bmatrix}}$$

$$\boxed{T = \exp\left(\xi^\wedge\right)}$$

$$\boxed{\exp\left(\xi^\wedge\right) = \begin{bmatrix} \exp(\omega^\wedge) & J(\omega)v \\ 0 & 1 \end{bmatrix}}$$

$$\boxed{T_e = T_d^{-1} T}$$

$$\boxed{e_T = \log\left(T_d^{-1} T\right)^\vee}$$

$$\boxed{\mathrm{Ad}_T = \begin{bmatrix} R & p^\wedge R \\ 0 & R \end{bmatrix}}$$

$$\boxed{\xi_s = \mathrm{Ad}_T\, \xi_b}$$

51 Study Problems

Problem 1

Show that if

$$R^T R = I,$$

then

$$R^{-1} = R^T.$$

Problem 2

For

$$\omega = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix},$$

construct

$$\omega^{\wedge}.$$

Then calculate

$$\omega^{\wedge} v$$

for

$$v = \begin{bmatrix} 4 \\ 5 \\ 6 \end{bmatrix}.$$

Verify that

$$\omega^{\wedge} v = \omega \times v.$$

Problem 3

Given

$$\omega = \begin{bmatrix} 0 \\ 0 \\ \theta \end{bmatrix},$$

derive

$$\exp\left(\omega^{\wedge}\right)$$

and show that it is the standard rotation matrix

$$R_z(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

Problem 4

Given

$$R_1, R_2 \in SO(3),$$

derive the relative rotation

$$R_1^T R_2.$$

Explain why this is preferable to the matrix difference

$$R_2 - R_1$$

for defining an intrinsic orientation error.

Problem 5

Given

$$T_1 = \begin{bmatrix} R_1 & p_1 \\ 0 & 1 \end{bmatrix}, \qquad T_2 = \begin{bmatrix} R_2 & p_2 \\ 0 & 1 \end{bmatrix},$$

derive

$$T_1 T_2$$

and

$$T_1^{-1}.$$

Problem 6

Given

$$T_d, T \in SE(3),$$

derive

$$e_T = \log\big(T_d^{-1} T\big)^{\vee}.$$

Separate the result into translational and rotational components.

Problem 7

Consider an RL policy

$$\xi_t = \pi_\theta(s_t).$$

Construct a geometric state transition using

$$T_{t+1} = T_t \exp\big(\Delta t\, \xi_t^{\wedge}\big).$$

Explain why the resulting pose remains in $SE(3)$.

Problem 8

Construct an RL reward using

$$e_T = \log\big(T_d^{-1} T\big)^{\vee}$$

and an action penalty

$$\|u\|^2.$$

Write the resulting objective.

Problem 9

Given

$$T = \begin{bmatrix} R & p \\ 0 & 1 \end{bmatrix},$$

derive

$$\mathrm{Ad}_T\,.$$

Verify that

$$\xi_s = \mathrm{Ad}_T\,\xi_b.$$

Problem 10

Design an RL environment in which:

1. the state contains an $SE(3)$ pose,
2. the policy outputs a six-dimensional twist,
3. the environment integrates the twist,
4. the reward depends on the logarithmic pose error.

Write the complete mathematical transition

$$s_t \to a_t \to T_{t+1} \to s_{t+1}.$$

52 Final Mathematical Picture

The essential geometric structure of robotic reinforcement learning is

Robot Configuration		Local Motion
$T \in SE(3)$	$\longleftrightarrow$	$\xi \in se(3)$
$R \in SO(3)$	$\longleftrightarrow$	$\omega \in so(3)$

with

$$\boxed{\xi \xrightarrow{\exp} T}$$

and

$$\boxed{T \xrightarrow{\log} \xi.}$$

For an RL controller,

$$\boxed{s_t \xrightarrow{\pi_\theta} \xi_t \xrightarrow{\exp} T_{t+1}.}$$

For a geometric observation,

$$\boxed{T_t \xrightarrow{T_d^{-1}} T_d^{-1} T_t \xrightarrow{\log} e_t.}$$

For a geometric reward,

$$\boxed{r_t = -\left\|\log\left(T_d^{-1}T_t\right)^{\vee}\right\|_2{}_Q^2 - \|u_t\|_2{}_R^2.}$$

For frame transformations,

$$\boxed{\xi_A = \mathrm{Ad}_{T_{AB}}\,\xi_B.}$$

These relations form the mathematical foundation for applying Lie-group methods to reinforcement-learning-based robotics.